

Weeklendar

A weekly displayed calendar.

Version 0.1.0

2026-09-16

<https://github.com/Yesteeer/typst-weeklendar>

1. Contents

2. Generate a calendar page	3
3. Add events	4
3.1. Add a single event	4
3.2. Early or late events	5
3.3. Events on multiple days	6
3.4. Periodic events	7
4. Margins and spacing	8
5. Customize title box	9
6. Customize days names	10
7. Customize event styling	11
8. Weeklendar's API	14
weeklendar	14
Parameters	14
starting-date	14
ending-date	14
height	15
width	15
margin	15
days	15
days-fct	15
days-pad	15
time-start	16
time-step	16
time-number	16
time-width	16
time-pad	16
title-fct	16
event-fct	16
debug	17
..events	17

2. Generate a calendar page

By default, calling the `weeklendar()` function with empty events generates an empty timetable of `datetime.today()`'s week. This is only meant to have access to an empty page and to the current's week dates, not to be used permanently, since it is based on today's datetime !

```
#import "local/weeklendar:0.1.0": *  
  
#weeklendar()
```

Week of 14 to 20 September 2026							
	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
08:00							
09:00							
10:00							
11:00							
12:00							
13:00							
14:00							
15:00							
16:00							
17:00							
18:00							

It is composed of a title box, which displays a default title as well as the starting and ending dates of the corresponding week. You can't change the style of this default title box, but you can completely customize the title with a custom function (see [Section 5](#)).

The timetable itself has an hourly timeline, which starts at "8:00" by default, and has 11 steps (i.e. it goes until 18:00). Those two parameters can be customized (`time-start` and `time-stepresp`). The horizontal spacings can be changed (see [Section 4](#)), but the vertical spacing between the lines and times can't be set manually and depends on the time numbers and the available space.

The days' name can also be customized (`days` argument). You can also change the number of days displayed, so you can remove week-ends if you are only interested in working days. In this case, only use events happening on the first 5 days to avoid events to exceed the timetable. Note that you can only remove days at the end of the week, otherwise the displayed events won't match with the right days ! The spacing above and below the days' names can also be customized (see [Section 4](#)).

By changing the `starting-date` and `ending-date` you can add more weeks to your calendar. These dates determine which events will be displayed, and not the range of the displayed weeks !

3. Add events

A calendar without events makes no sense ! So let us see how we can add events to our weeklendar. From now on, we will always fix starting and ending dates.

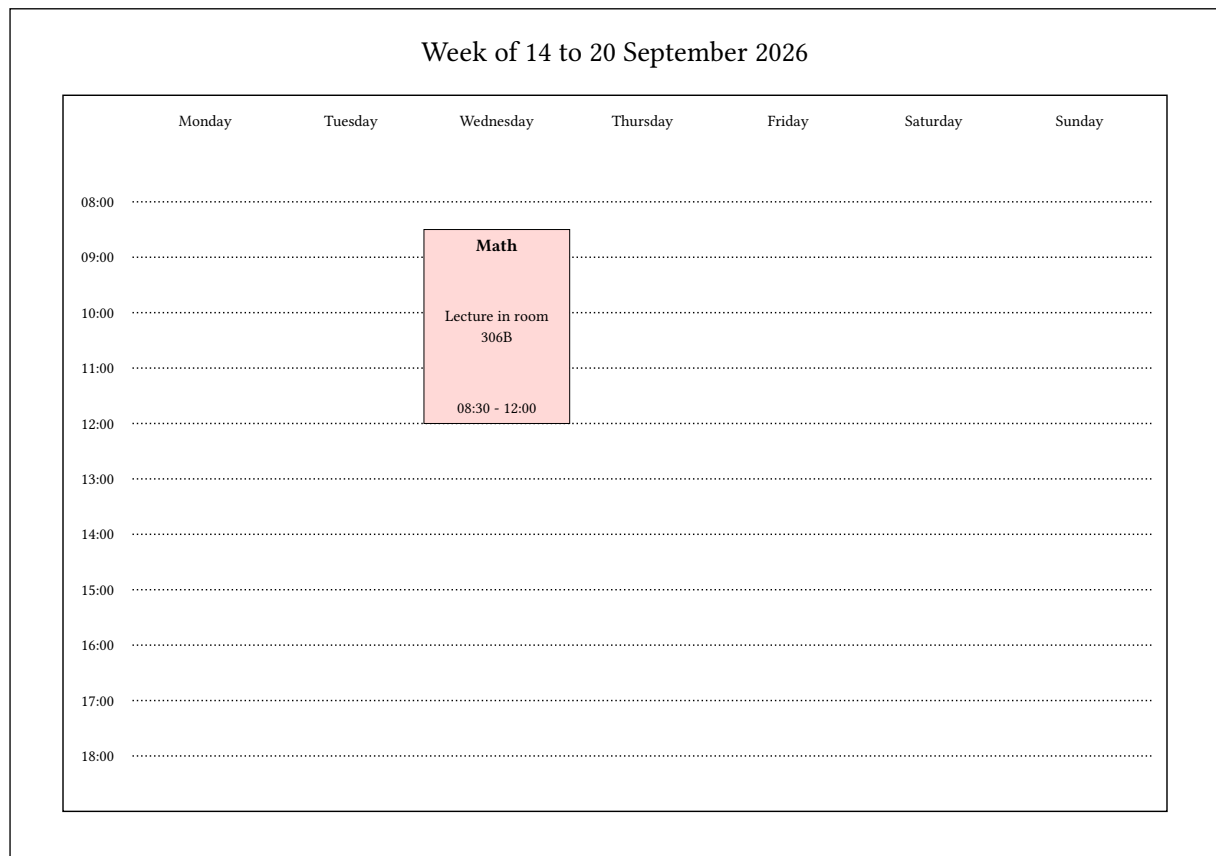
3.1. Add a single event

An event is characterized by a dictionary containing at least a start and end datetime. Those can be specified either as an ISO 8601 extended format and some variants (see [starting-date's API](#)) or directly as a complete datetime (but it seems painful to me to do it this way...).

```
#let event = (  
  start: "2026-09-16T08:30", // required  
  end: "2026-09-16T12",    // required  
  summary: "Math",         // optional with default: ""  
  description: "Lecture in room 306B", // optional with default: ""  
  fill: red.lighten(80%),   // optional with default: blue.lighten(70%)  
  repeat-until: none,      // optional with default: none  
  repeat-frequency: none,  // optional with default: none  
)
```

The first 3 optional keys allow you to customize the default appearance of the displayed event. We will discuss the 2 other optional keys when we look at periodic events (see [Section 3.4](#)). We will see in [Section 7](#) that we can add further entries to an event's dictionary. Let's see what happens if we add this event to `weeklendar()`. From now on we will omit the package's import in the examples.

```
#weeklendar(starting-date: "2026-09-14", ending-date: "2026-09-20", event)
```



It could happen that an event doesn't fit in the displayed timelines. Either you add more timelines, as explained in [Section 2](#), or you let the event exceed the timelines. The height of the displayed event's rectangle expands until 1 hour before the first one and 1 hour after the last one linearly and then it stops, even if the event starts earlier or ends later. Let us illustrate this with an example:

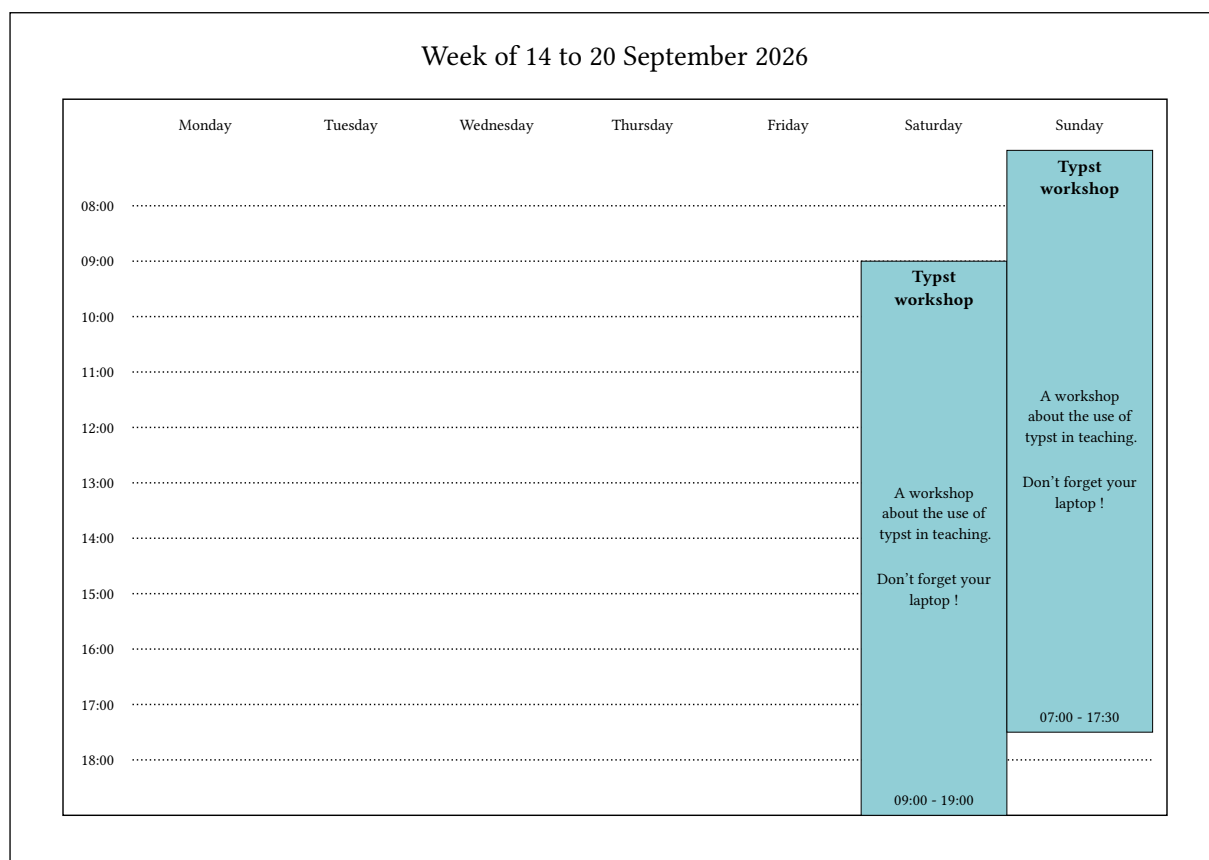
Week of 14 to 20 September 2026

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
08:00			Early morning walk 06:30 - 08:30			Morning walk 07:30 - 09:30	
09:00							
10:00							
11:00							
12:00							
13:00							
14:00							
15:00							
16:00							
17:00							
18:00					Dinner Bring a salad ! 17:30 - 21:00		

3.3. Events on multiple days

Weeklendar also manages events that span on multiple days. Let's suppose you want to add a two-day Typst workshop to your calendar. Then you add a single event with corresponding starting and ending datetimes and weeklendar does the job !

```
#let event = (  
  start: "2026-09-19T09:00",  
  end: "2026-09-20T17:30",  
  summary: "Typst workshop",  
  description: [  
    #v(1em)  
    A workshop about the use of typst in teaching.  
    #v(1em)  
    Don't forget your laptop !  
  ],  
  fill: rgb("#239dad").lighten(50%)  
)  
  
#weeklendar(starting-date: "2026-09-14", ending-date: "2026-09-20", event)
```



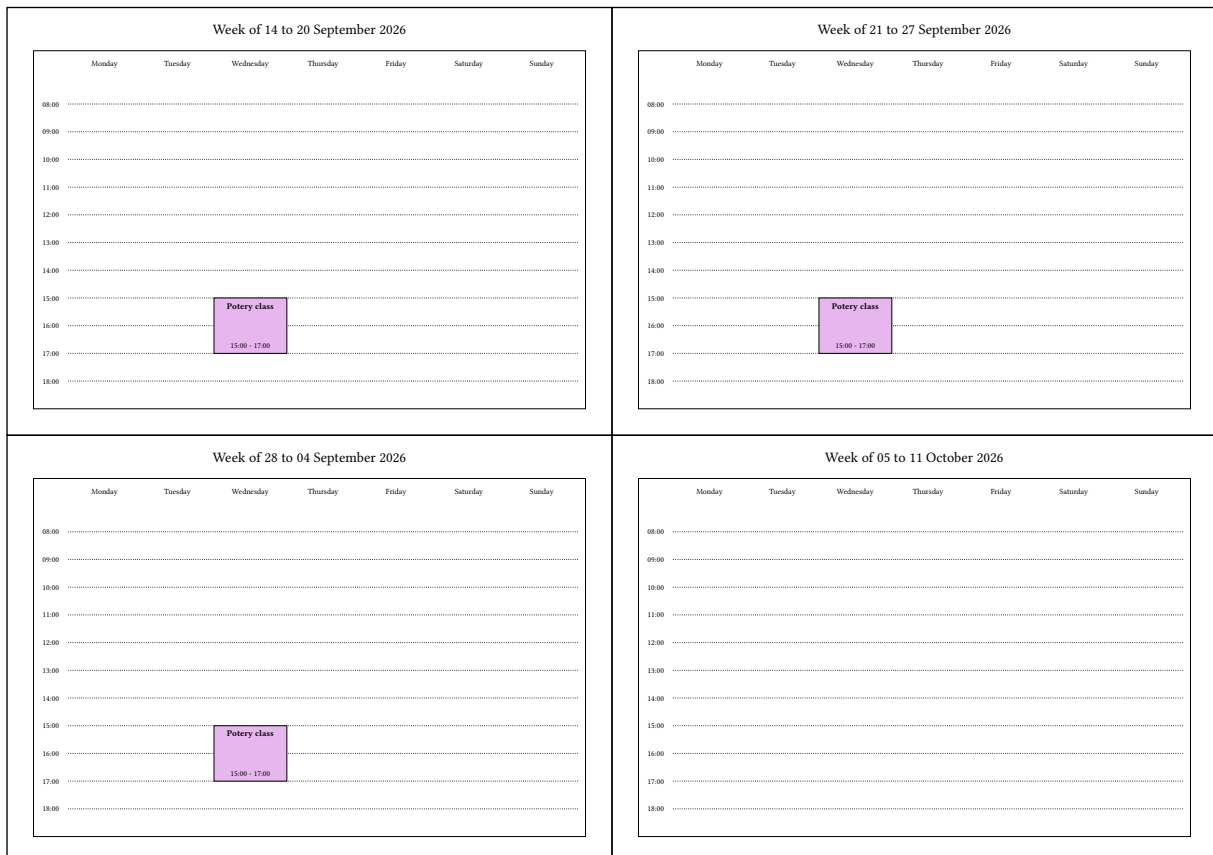
For now, the description appears on both (or all) days of a multiple days event. This is for practical reasons, if an event spans on two weeks so that the information appears on both week's timetable.

Also, when an event spans over more multiple days, the default timespans appearing on the event's boxes end 1 hour after the last timeline (in this case "19:00") and start 1 hour before the first timeline (here "7:00").

3.4. Periodic events

Periodic events are fairly common and it would be rather tedious to add the events one by one. Instead, `weeklendar` allows you to use the optional keys `repeat-until` and `repeat-frequency` to automatically generate multiple instances of a periodic event. In order to add periodic events, both of those parameters should be provided. `repeat-until` allow you to specify a datetime (given as an ISO 8601 str or a datetime) until which the given event should be repeated and `repeat-frequency` the frequency of the event (given as a duration). Let's see an example:

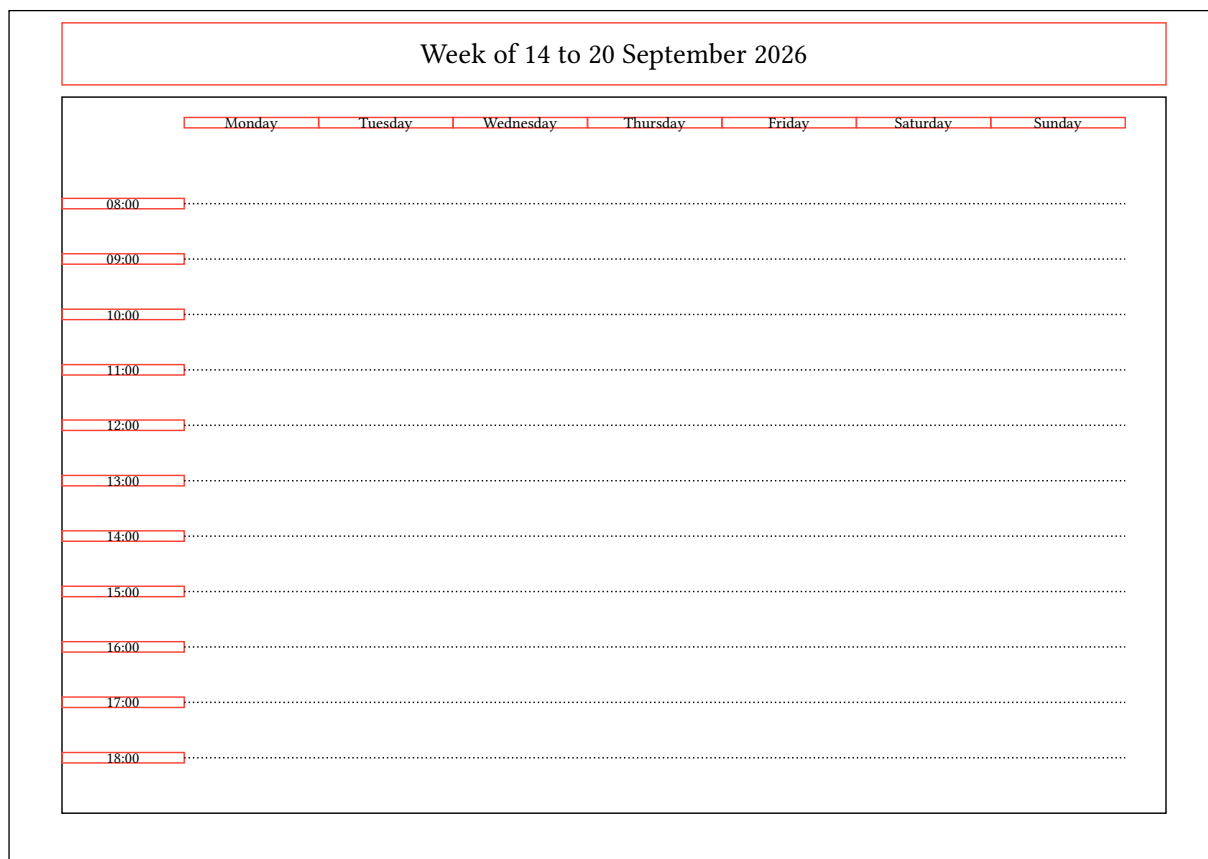
```
#let event = (  
  start: "2026-09-16T15",  
  end: "2026-09-16T17",  
  summary: "Potery class",  
  fill: purple.lighten(70%),  
  repeat-until: "2026-10-05",  
  repeat-frequency: duration(days: 7)  
)  
  
#weeklendar(starting-date: "2026-09-14", ending-date: "2026-10-11", event)
```



As you can see, a single event is repeated until the given date, then it stops. If `weeklendar()`'s ending-date is sooner than the repeat-until date, then those repeated events that come after the ending-date won't appear.

4. Margins and spacing

Let us now see how we can customize spacing and margin of the timetable. First we can change the margins around the title and timetable. The left, right and bottom margins work as expected, since they correspond to the distance between the page's borders and the timetable's borders. The top margin however, is the space that appears at the top **and** the bottom of the title's box. By calling `weeklendar` (debug: `true`) one can visualize how these work:



On the example above, the following default margins are used: (left: `1.3cm`, right: `1.3cm`, top: `0.3cm`, bottom: `1.3cm`). Those defaults don't appear in the API, but are later combined with the `weeklendar`'s margin argument, so that margins can be individually overwritten.

On the debug timetable, the red box around the times have width `time-width + 2*time-pad` (left and right padding). This same `time-pad` is added at the end of each timeline. Their height is determined by their content.

Finally, the padding above and below the days' names can be changed via the `days-pad` argument, with defaults: (above: `0.5cm`, below: `0.5cm`). Note that `days-pad.below` gives the space between the bottom of the days' debug-box (which height is based on its content and width on the length of the timelines) and the top of an event which starts at "7:00" (and not to the first timeline). That explains why the space between the days' names and the first timeline is bigger than the spacing above, although the padding is the same.

5. Customize title box

The title and subtitle of a week's timetable can't be changed. However, you can provide `weeklendar()` as custom title-fct function to generate you own title boxes. The only constraint are that the function must take the first week's monday datetime as an argument and that its margins are determined by the margin argument. The default function that is used is the following:

```
#let default-title-fct(monday) = {  
  grid(  
    columns: 1fr,  
    align: center,  
    inset: 15pt,  
    [  
      #set text(20pt)  
      Week of #(monday).display("[day]") to #(monday + duration(days:  
6)).display("[day]") #monday.display("[month repr:long]") 2026  
    ],  
  )  
}
```

We can for example only show the month and the week number.

```
#let title(monday) = (  
  grid(  
    columns: 1fr, // important to make the content really centered  
    align: center,  
    row-gutter: 15pt,  
    [  
      #set text(20pt)  
      #monday.display("[month repr:long] [year]")  
    ],  
    [Week number: #monday.display("[week_number]")]  
  )  
)  
  
#weeklendar(  
  time-number: 2,  
  height: 8cm,  
  weekly-title: title,  
)
```

September 2026						
Week number: 38						
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
08:00						
09:00						

6. Customize days names

The previous example looks good, but it would be nice to have the dates of the week's days somewhere. What about next to or below the days names ? That is the content of this section

With the same idea, we can also customize the days appearance in the timetable by providing our own `days-fct` function. This function should take two arguments: the date of the week's monday, and the day's name to display. The default function only uses the former argument:

```
#let default-days-fct(monday, day-list, day-number) = {  
  grid(  
    columns: 1fr, // important to make the content really centered  
    align: center,  
    [#day-list.at(day-number)],  
  )  
}
```

But let's say that we want to display the day's date below its name, then we can complete our previous example as follows:

```
#let title(monday) = [...]  
  
#let days(monday, day-list, day-number) = {  
  grid(  
    columns: 1fr, // important to make the content really centered  
    align: center,  
    row-gutter: 10pt,  
    [#day-list.at(day-number)],  
    [#(monday + duration(days: day-number)).display("[day]")#super("th")]  
  )  
}  
  
#weeklendar(  
  time-number: 2,  
  height: 8cm,  
  title-fct: title,  
  days-fct: days,  
)
```

September 2026						
Week number: 38						
Monday 14 th	Tuesday 15 th	Wednesday 16 th	Thursday 17 th	Friday 18 th	Saturday 19 th	Sunday 20 th
08:00						
09:00						

7. Customize event styling

Finally, the event's boxes can also be fully customized. If you're not happy with the default layout, you can provide your own `event-fct` which takes an event dictionary as an argument. The default function is defined as:

```
#let default-event-fct(event) = {
  rect(
    width: 100%,
    height: 100%,
    fill: event.fill,
    stroke: black + .5pt,
  ) [
    #grid(
      columns: 1fr, // makes content really centered
      rows: 1fr, // makes the event's box take all the allocated vertical space
      inset: 5pt,
      align: (x, y) => {
        if y == 0 { top + center }
        else if y == 1 { horizon + center }
        else { bottom + center }
      },
      [
        #text(size: 1.1em) [*#event.summary*]
      ],
      [
        #event.description
      ],
      [
        #event.start.display("[hour]:[minute]") - #event.end.display("[hour]:
[minute]")
      ]
    )
  ]
}
```

The important thing to keep in mind is that whatever content you build for your event, make sure that it takes the whole allocated vertical and horizontal space. Otherwise, the events won't appear properly. The nice thing here, is that you can provide additional keys to your events, which will then be accessible to your custom `event-fct` function !

As an example, we will use the [Showybox](#) package to customize our events. We will keep things simple, but the customization possibilities are endless. Here we simply want to add the possibility to remove the hours when not necessary, so each of our events will have to provide a `show-hours` key. The rest is just re-styling.

```

#import "local/weeklendar:0.1.0": *
#import "@preview/showybox:2.0.4": showybox

#let events = (
  (
    start: "2026-09-14T12:18", end: "2026-09-14T14:18", summary: "Hairdresser",
    fill: olive, show-hours: true,
  ),
  (
    start: "2026-09-17T06:15", end: "2026-09-17T09:45", summary: "Morning run",
    fill: maroon, show-hours: true,
  ),
  (
    start: "2026-09-19T08:00", end: "2026-09-20T17:00", summary: "Mountain trip",
    description: [
      Don't forget to take
      - sunscreen
      - cap
      - pic-nic
      - ...
    ], fill: eastern, show-hours: false,
  ),
)

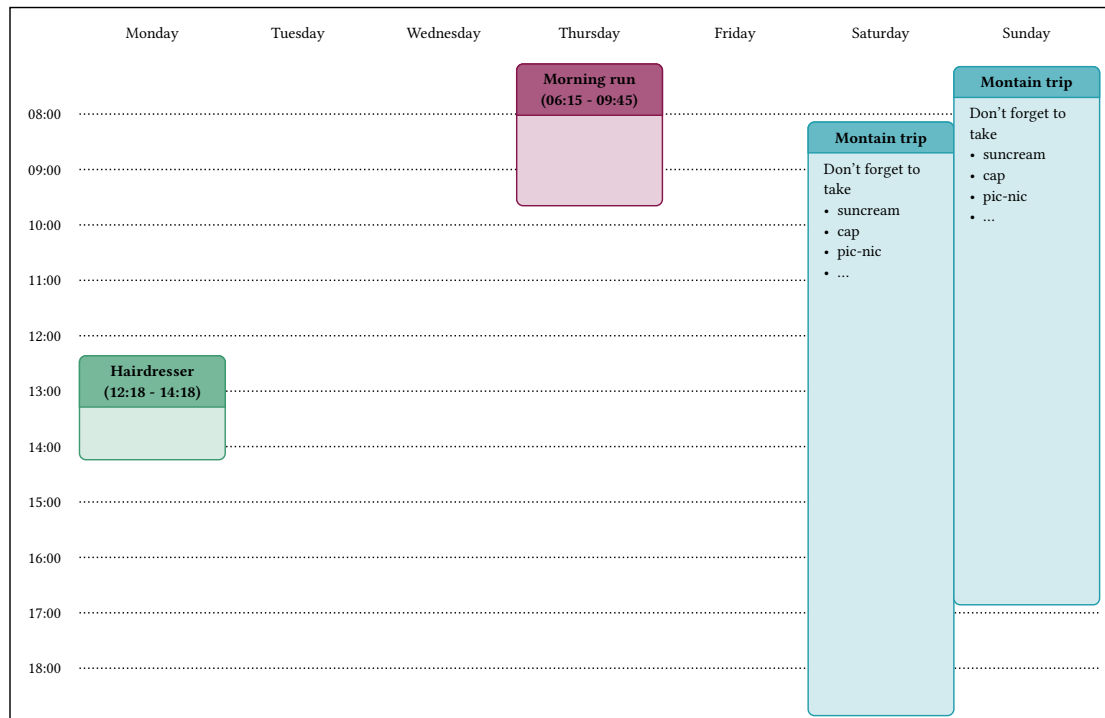
#let event-fct(event) = {
  let title = if event.show-hours [
    #event.summary \
    (#event.start.display("[hour]:[minute]") - #event.end.display("[hour]:[minute]"))
  ] else {event.summary}

  showybox(
    title: title,
    footer-style: (
      align: center,
    ),
    title-style: (
      weight: 900,
      color: black,
      align: center
    ),
    frame: (
      border-color: event.fill,
      title-color: event.fill.lighten(30%),
      body-color: event.fill.lighten(80%),
    ),
  )[
    #event.description
    #v(1fr)
  ]
}

#weeklendar(
  starting-date: "2026-09-14", ending-date: "2026-09-20", event-fct: event-fct, ..events
)

```

Week of 14 to 20 September 2026



8. Weeklendar's API

- `weeklendar()`

weeklendar

This function generates a weekly calendar from a starting date to a end date. It displays events, which are parametrized by a dictionary.

Parameters

```
weeklendar(  
    starting-date: str,  
    ending-date: str,  
    height: length,  
    width: length,  
    margin: dictionary,  
    days: array,  
    days-fct: auto function,  
    days-pad: dictionary,  
    time-start: datetime,  
    time-step: duration,  
    time-number: int,  
    time-width: length,  
    time-pad: length,  
    title-fct: auto function,  
    event-fct: auto function,  
    debug: boolean,  
    ..events: array  
) -> content
```

starting-date str

The starting-date defines the first week to be displayed in the calendar.

The date must be given with the following format (based on the ISO 8601 extended format):

- “dd-mm-yyyy”
- “dd-mm-yyyyThh”
- “dd-mm-yyyyThh:mm”
- “dd-mm-yyyyThh:mm:ss”

Or directly as a datetime element.

Default: `datetime.today()`

ending-date str

The ending date defines the last week to be displayed in the calendar.

The accepted date format are the same as for the starting-date.

Default: `datetime.today() + duration(days: 7)`

height length

The paper's height (not the same as the timetable's height !).

Default: 21cm

width length

The paper's width (not the same as the timetable's width !).

Default: 29.70cm

margin dictionary

The paper's margins, which define the timetable's height and width, together with the title's dimensions.

The top margin is applied above and below the title, so that the total distance between the top of the timetable and the top of the page is equal to the title's height + 2 * margin.top.

Default: (:)

days array

The days names to be displayed on the timetable.

Default: (

```
"Monday",  
"Tuesday",  
"Wednesday",  
"Thursday",  
"Friday",  
"Saturday",  
"Sunday"
```

)

days-fct auto or function

A function to customize the displayed days appearance. It takes a day name and the first week's monday date as arguments.

Default: auto

days-pad dictionary

The padding below and above the displayed days.

Default: (:)

time-start `datetime`

The first timeline to appear on the timetable.

Default: `datetime(hour: 8, minute: 0, second: 0)`

time-step `duration`

The step between two successive displayed timelines.

Default: `duration(hours: 1)`

time-number `int`

The number of timelines.

The spacing between consecutive timelines depends on this number.

Default: `11`

time-width `length`

The width of the box containing the timeline's times.

Default: `1cm`

time-pad `length`

The padding on the left/right around the timeline's time (added to the time's width) and at the end of the timeline's line.

Default: `10pt`

title-fct `auto` or `function`

A custom function for generating weekly titles. It takes the first week's day "monday" as an argument and build a custom title depending on monday's date.

Default: `auto`

event-fct `auto` or `function`

A custom function for the generic content of an event. It takes an event as an argument. It is recommended to build a content that expands vertically (for example by using `#v(1fr)`), so that the events are displayed on the entire associated timeline.

Default: `auto`

debug `boolean`

A debugging option to show some of the displayed element's boundaries.

Default: `false`

..events `array`

Events to be displayed on the calendar, passed as a positional arguments.

Each event is described by a dictionary with two mandatory arguments: `start` and `end`, which are to be given in the ISO 8601 extended format (see `starting-date` above) or directly as a datetime element.

If you want to make a periodic event, then you can use `repeat-until` and `repeat-frequency`. You can add extra named arguments, that can be accessed by your custom `event-fct`.